浙江大学
ZHEJIANG UNIVERSITY

# RISC-V 浮点加操作指令及硬件扩展设计

# 实验报告


姓 名 吴溪语

学 号 3230103327


实验日期 2025.12.8

指导老师 赵武锋

# 1 实验目的

本实验旨在为开源 VeeRwolf 项目的 EL2 处理器扩展单精度浮点加功能。EL2 原生支持 RV32IMC 指令集，本实验需遵循 IEEE 754 浮点数标准，完成浮点加指令在译码级的配置、`fp_adder` 模块与整数除法器的集成，并通过仿真工具验证其功能正确性。同时，通过实验过程掌握 Linux 开发环境的使用方法以及 RISC-V 处理器硬件功能扩展的核心流程。

# 2 实验任务与要求

(1). 搭建 WSL2 系统与 Ubuntu 环境，安装 VS Code 相关扩展，挂载工程目录并配置 Catapult Studio、Verilator、GTKWave 等工具。

(2). 修改 `el2_def.sv` 等译码级文件，新增浮点信号与指令定义，生成并替换译码方程，完善模块间信号连接。

(3). 自定义将 `fp_adder` 模块集成到除法器中，处理信号锁存与时延问题。

(4). 编译仿真文件，通过测试程序生成波形，验证浮点加法及除法指令功能。

(5). 新增浮点乘除软硬件支持，使用 Catapult Studio 编写并测试自定义文件。

# 3 实验过程

## 3.1 环境配置

环境搭建的具体步骤已在指导文件中详细给出，这里仅对整体框架和关键要点进行简要概括：

(1). **安装 WSL2：** 以管理员身份打开 PowerShell，执行 `wsl --install` 安装 Ubuntu 发行版。通过 `wsl --list` 验证安装结果，后续使用 `wsl` 进入系统，并完成软件源更新及基础工具的安装。

(2). **配置 VS Code：** 安装 WSL 与 Remote Explorer 扩展，连接 Ubuntu 虚拟机环境。挂载 `RVfpga_NexysA7-DDR` 工程目录，要求路径中不包含中文，并将路径格式转换为 `/mnt/盘符/...`。

(3). **安装相关工具：** 根据需要选择性安装 Catapult Studio；必须安装 Verilator（含相关依赖）与 Gtkwave。分别通过对应命令完成安装，并运行相关指令验证安装是否成功。

## 3.2 译码级及信号修改

这一部分主要是根据说明文件来做，具体步骤如下：

(1) 创建一个名为 fp_pkt_t 的新结构类型，其中包含 3 个比特：fp_add、fp_mul 和 fp_div；这些比特分别表示处理器是否正在执行浮点加法、浮点乘法或浮点除法。

```
typedef struct packed {
                    logic fp_add;
                    logic fp_mul;
                    logic fp_div;
                  }fp_pkt_t;
typedef struct packed {
                  logic  trace_rv_i_valid_ip;
                  logic [31:0] trace_rv_i_insn_ip;
                  logic [31:0] trace_rv_i_address_ip;
                  logic  trace_rv_i_exception_ip;
                  logic [4:0] trace_rv_i_ecause_ip;
                  logic  trace_rv_i_interrupt_ip;
                  logic [31:0] trace_rv_i_tval_ip;
                  } el2_trace_pkt_t;
```

(2) 创建三个新的 bits，分别名为 fp_add、fp_mul 和 fp_div。

```
typedef struct packed {
                    logic fp_add;
                    logic fp_mul;
                    logic fp_div;
                    logic clz;
                    logic ctz;
                    logic cpop;
                    logic sext_b;
                    logic sext_h;
```

(3) 在.definition 段，添加三个新指令的定义.

```
.definition

fadd =    [0000000..................1010011]
fmul =    [0001000..................1010011]
fdiv =    [0001100..................1010011]

clz       =  [011000000000.....001.....0010011]
ctz       =  [011000000001.....001.....0010011]
cpop      =  [011000000010.....001.....0010011]
sext_b    =  [011000000100.....001.....0010011]
sext_h    =  [011000000101.....001.....0010011]

min       =  [0000101..........100.....0110011]
max       =  [0000101..........110.....0110011]
minu      =  [0000101..........101.....0110011]
maxu      =  [0000101..........111.....0110011]
```

(4) 在.output 段，为每条指令创建一个新的 bit。

```
rv32i = {
    fp_add
    fp_mul
    fp_div
    alu
    rs1
    rs2
    imm12
```

(5) 在 dec 目录执行命令生成译码方程文件，替换 el2_dec_decode_ctl.sv 中旧方程。

(6) 在模块 e12_dec_decode_ctl 中，使用信号 i0_dp 为信号 fp_p 中的新浮点控制位赋值。

```
// FP
assign fp_p.fp_add = i0_dp.fp_add;
assign fp_p.fp_mul = i0_dp.fp_mul;
assign fp_p.fp_div = i0_dp.fp_div;
```

(7) 在 el2_dec_decode_ctl.sv 的开头输出信号中，加入 fp_p。

```
output el2_div_pkt_t    div_p,
output fp_pkt_t         fp_p,
output logic [4:0]       div_waddr_wb,
output logic             dec_div_cancel,
```

(8) 在 el2_dec.sv 的输出信号中, 加入 fp_p。

```
output el2_lsu_pkt_t    lsu_p,
output logic             dec_qual_lsu_d,
output el2_mul_pkt_t    mul_p,
output el2_div_pkt_t    div_p,
output fp_pkt_t fp_p,
output logic             dec_div_cancel,
```

(9) 在 el2_exu.sv 的输入信号中，加入 fp_p。

```
input logic          dec_qual_lsu_d,
input el2_mul_pkt_t mul_p,
input el2_div_pkt_t div_p,
input fp_pkt_t       fp_p,
input logic          dec_div_cancel,
```

(10) 在 src/VeeRwolf/VeeR_EL2CoreComplex/el2_veer.sv 的连接信号中，加入 fp_p。

```
el2_mul_pkt_t  mul_p;
fp_pkt_t       fp_p;
el2_div_pkt_t  div_p;
logic           dec_div_cancel;
```

(11) 在 el2_exu.sv 文件中，修改除法器例化，新增端口 fp。对应的，修改 el2_exu_div_ctl.sv 文件。

```
module el2_exu_div_ctl
import el2_pkg::*;
#(
`include "el2_param.vh"
)
  (
   input logic           clk,
   input logic           rst_l,
   input logic           scan_mode,

   input el2_div_pkt_t  dp,
   input fp_pkt_t       fp,
   input logic  [31:0]   dividend,
   input logic  [31:0]   divisor,

   input logic           cancel,


   output logic          finish_dly_div_fp,
   output logic [31:0]   out
  );
```

## 3.3 fp_adder 集成以及 fp_mul 和 fp_div 集成

在对 el2_exu_div_ctl 修改时，主要就是集成了 fpadd 和 fpmul 以及 fpdiv，在原有除法控制模块基础上，新增浮点加、乘、除运算支持。

具体的修改代码如下：

```
module el2_exu_div_ctl
import el2_pkg::*;
#(
`include "el2_param.vh"
)
(
   input logic           clk,                      // Top level clock
   input logic           rst_l,                    // Reset
   input logic           scan_mode,                // Scan mode

   input el2_div_pkt_t  dp,                       // valid, sign, rem
   input fp_pkt_t       fp,
   input logic  [31:0]   dividend,                 // Numerator
   input logic  [31:0]   divisor,                  // Denominator

   input logic           cancel,                   // Cancel divide

   output logic          finish_dly_div_fp,        // Finish to match data
   output logic [31:0]   out                       // Result
  );

   logic [31:0]          out_raw;

   if (pt.DIV_NEW == 0) begin
        el2_exu_div_existing_1bit_cheapshortq   i_existing_1bit_div_cheapshortq (
            .clk(clk), .rst_l(rst_l), .scan_mode(scan_mode), .cancel(cancel),
            .valid_in(dp.valid), .signed_in(~dp.unsign), .rem_in(dp.rem),
            .dividend_in(dividend), .divisor_in(divisor),
            .valid_out(finish_dly), .data_out(out_raw)
        );
   end else if ( (pt.DIV_NEW == 1) & (pt.DIV_BIT == 1) ) begin
        el2_exu_div_new_1bit_fullshortq         i_new_1bit_div_fullshortq (
            .clk(clk), .rst_l(rst_l), .scan_mode(scan_mode), .cancel(cancel),
            .valid_in(dp.valid), .signed_in(~dp.unsign), .rem_in(dp.rem),
            .dividend_in(dividend), .divisor_in(divisor),
            .valid_out(finish_dly), .data_out(out_raw)
        );
   end else if ( (pt.DIV_NEW == 1) & (pt.DIV_BIT == 2) ) begin
        el2_exu_div_new_2bit_fullshortq         i_new_2bit_div_fullshortq (
            .clk(clk), .rst_l(rst_l), .scan_mode(scan_mode), .cancel(cancel),
            .valid_in(dp.valid), .signed_in(~dp.unsign), .rem_in(dp.rem),
            .dividend_in(dividend), .divisor_in(divisor),
            .valid_out(finish_dly), .data_out(out_raw)
        );
   end else if ( (pt.DIV_NEW == 1) & (pt.DIV_BIT == 3) ) begin
```

```
        el2_exu_div_new_3bit_fullshortq         i_new_3bit_div_fullshortq (
            .clk(clk), .rst_l(rst_l), .scan_mode(scan_mode), .cancel(cancel),
            .valid_in(dp.valid), .signed_in(~dp.unsign), .rem_in(dp.rem),
            .dividend_in(dividend), .divisor_in(divisor),
            .valid_out(finish_dly), .data_out(out_raw)
        );
    end else if ( (pt.DIV_NEW == 1) & (pt.DIV_BIT == 4) ) begin
        el2_exu_div_new_4bit_fullshortq         i_new_4bit_div_fullshortq (
            .clk(clk), .rst_l(rst_l), .scan_mode(scan_mode), .cancel(cancel),
            .valid_in(dp.valid), .signed_in(~dp.unsign), .rem_in(dp.rem),
            .dividend_in(dividend), .divisor_in(divisor),
            .valid_out(finish_dly), .data_out(out_raw)
        );
    end

    // --- Control Signals Mapping ---
    wire fp_add_request = fp.fp_add;
    wire fp_mul_request = fp.fp_mul;
    wire fp_div_request = fp.fp_div;

    // --- Delay Pipeline & Data Registers ---
    // ADD Path
    reg         fp_add_request_d, fp_add_request_dd;
    reg [31:0]  fp_add_dividend_d, fp_add_dividend_dd;
    reg [31:0]  fp_add_divisor_d,  fp_add_divisor_dd;

    // MUL Path
    reg         fp_mul_request_d, fp_mul_request_dd;
    reg [31:0]  fp_mul_dividend_d, fp_mul_dividend_dd;
    reg [31:0]  fp_mul_divisor_d,  fp_mul_divisor_dd;

    // DIV Path
    reg         fp_div_request_d, fp_div_request_dd;
    reg [31:0]  fp_div_dividend_d, fp_div_dividend_dd;
    reg [31:0]  fp_div_divisor_d,  fp_div_divisor_dd;

    // --- Functional Unit Interfaces ---
    wire [31:0] fp_adder_out;
    wire        fp_adder_done;

    wire [31:0] fp_mul_out;
    wire        fp_mul_done;

    wire [31:0] fp_div_out;
    wire        fp_div_done;

```

```
    // --- Result Latches ---
    reg [31:0]  fp_add_result_reg;
    reg [31:0]  fp_mul_result_reg;
    reg [31:0]  fp_div_result_reg;

    always @(posedge clk or negedge rst_l) begin
        if (!rst_l) begin
            fp_add_request_d <= 1'b0; fp_add_request_dd <= 1'b0;
            fp_mul_request_d <= 1'b0; fp_mul_request_dd <= 1'b0;
            fp_div_request_d <= 1'b0; fp_div_request_dd <= 1'b0;
            fp_add_result_reg <= '0;
            fp_mul_result_reg <= '0;
            fp_div_result_reg <= '0;
        end else begin
            fp_add_request_d  <= fp_add_request;
            fp_add_request_dd <= fp_add_request_d;
            fp_mul_request_d  <= fp_mul_request;
            fp_mul_request_dd <= fp_mul_request_d;
            fp_div_request_d  <= fp_div_request;
            fp_div_request_dd <= fp_div_request_d;

            if (fp_adder_done) fp_add_result_reg <= fp_adder_out;
            if (fp_mul_done)   fp_mul_result_reg <= fp_mul_out;
            if (fp_div_done)   fp_div_result_reg <= fp_div_out;
        end
    end

    wire [3:0] active_operation;
    wire add_operation_active     = active_operation[0];
    wire mul_operation_active     = active_operation[1];
    wire div_operation_active     = active_operation[2];
    wire int_div_operation_active = active_operation[3];

    fp_operation_latch op_latch (
        .clk(clk), .rst_l(rst_l), .dp_valid(dp.valid),
        .fp_add(fp.fp_add), .fp_mul(fp.fp_mul), .fp_div(fp.fp_div),
        .active_op(active_operation)
    );

    always @(*) begin
        out = 32'b0;
        case (1'b1)
            (int_div_operation_active && finish_dly) : out = out_raw;
            (add_operation_active && fp_adder_done)  : out = fp_add_result_reg;
            (mul_operation_active && fp_mul_done)    : out = fp_mul_result_reg;
            (div_operation_active && fp_div_done)    : out = fp_div_result_reg;
```

```
            default: out = 32'b0;
        endcase
    end

    assign finish_dly_div_fp =
        (int_div_operation_active && finish_dly) ||
        (add_operation_active     && fp_adder_done) ||
        (mul_operation_active     && fp_mul_done) ||
        (div_operation_active     && fp_div_done);

endmodule


module fp_operation_latch (
    input wire clk,
    input wire rst_l,
    input wire dp_valid,
    input wire fp_add,
    input wire fp_mul,
    input wire fp_div,
    output reg [3:0] active_op
);

    localparam OP_ADD  = 4'b0001;
    localparam OP_MUL  = 4'b0010;
    localparam OP_DIV  = 4'b0100;
    localparam OP_IDIV = 4'b1000;

    wire any_fp_request = fp_add | fp_mul | fp_div;

    always @(posedge clk or negedge rst_l) begin
        if (!rst_l)
            active_op <= OP_IDIV;
        else if (dp_valid) begin
            if (any_fp_request) begin
                if      (fp_div) active_op <= OP_DIV;
                else if (fp_mul) active_op <= OP_MUL;
                else if (fp_add) active_op <= OP_ADD;
                else             active_op <= OP_IDIV;
            end else
                active_op <= OP_IDIV;
        end
    end
endmodule
```

Listing 1: el2_exu_div_ctl 与 fp_operation_latch 完整实现

### 3.4 新的测试文件 test.S

新的 test.S 用于测试浮点乘法和浮点除法的正确性，测试样例中选取了 5.0 和 2.5 分别进行乘法和除法。下面是测试文件代码:

```
/* 定义延时宏 */
#define DELAY_BLOCK_4      nop; nop; nop; nop;
#define DELAY_BLOCK_10     DELAY_BLOCK_4 DELAY_BLOCK_4 nop; nop;

.globl main
main:
    # ------------------------------------------------
    # 初始化浮点操作数 (IEEE 754 单精度标准)
    # ------------------------------------------------
    # t3 = 2.5 (Hex: 0x40200000)
    li t3, 0x40200000
    # t4 = 5.0 (Hex: 0x40a00000)
    li t4, 0x40a00000

TEST_LOOP:
    # [测试点 A] 浮点乘法: 5.0 * 2.5 = 12.5
    # 预期结果: 0x41480000
    DELAY_BLOCK_4
    .word 0x11ce8f53  # fmul.s 指令码

    # [测试点 B] 浮点除法: 5.0 / 2.5 = 2.0
    # 预期结果: 0x40000000
    DELAY_BLOCK_4
    .word 0x19ce8f53  # fdiv.s 指令码

    DELAY_BLOCK_10
    beq zero, zero, TEST_LOOP # 保持循环以观察波形
```

Listing 2: test.S 完整代码

# 4 实验结果与分析

### 4.1 fpadd.S 的测试结果

终端中通过 cd 命令进入工程目录 RVfpga_NexysA7-DDR/Labs/Lab18/commandLine，执行如下仿真命令:

```
./Simulators/verilatorSIM_Trace/Vrvfpgasim \
+ram_init_file=fp_add.vh +vcd=1
```

仿真运行约 3 秒后，按下 `Ctrl+C` 终止执行。此时可在当前目录下生成 `trace.vcd` 文件，该文件为标准 VCD 波形文件，可供 GTKWave 工具读取。

随后在终端中运行：

```
gtkwave trace.vcd
```

即可打开波形显示器。在 GTKWave 中，通过左上角的 `File -> Read TCL Script` 读取预先设置的信号脚本 `test.tcl`，即可加载关键信号并进行波形观察。下面是测试结果波形图：

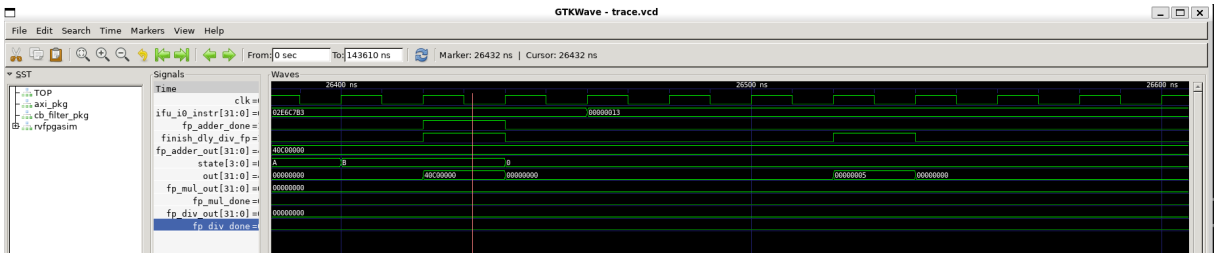


Figure 1: fpadd.S 的测试结果

如图所示，在 IFU 取指阶段，指令 `0x01CE8F53` 表示浮点加法（`fp_add`）指令，`0x00000013` 为插入的空指令（NOP）。在该测试中，浮点加法模块输入的两个操作数分别为 `0x40800000` 与 `0x40000000`，对应的浮点数相加结果在 `out` 信号上正确产生 `0x40C00000`。同时，运算结束后 `finish` 信号被正确拉高，表明浮点加法单元能够正确完成计算并返回结果。

## 4.2 test.S 的测试结果

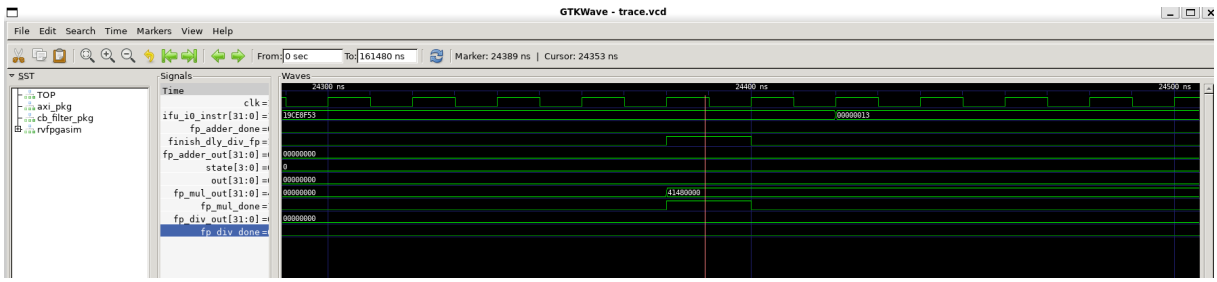


Figure 2: 浮点乘法波形图

`test.S` 主要用来验证浮点乘除法的功能。为了防止流水线冲突，也为了方便在波形图上观察，我在指令之间插入了一些 `nop` 延时。初始化时，我将寄存器 `t3` 和 `t4` 分别设为 `0x40200000`（2.5）和 `0x40A00000`（5.0）。

从仿真波形可以看出：在执行浮点乘法（指令 `0x11CE8F53`）计算 $5.0 \times 2.5$ 时，`out` 信号输出了 `0x41480000`（即 12.5），结果正确；在执行浮点除法（指令 `0x19CE8F53`）计

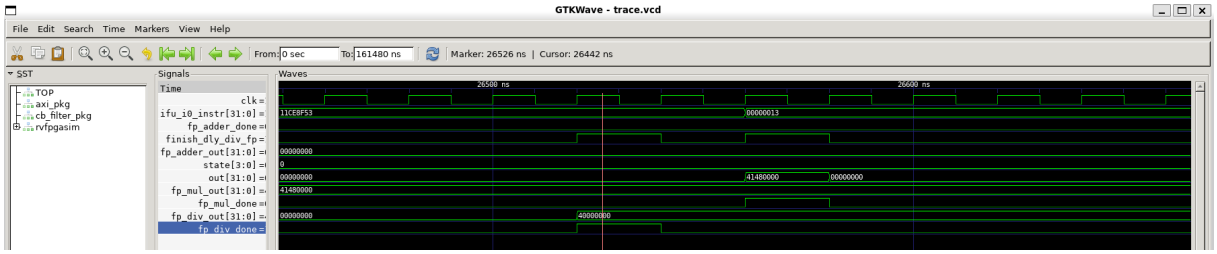


Figure 3: 浮点除法波形图

算 $5.0 \div 2.5$ 时，结果也正确输出了 `0x40000000`（即 2.0）。两次运算结束后，`finish` 信号均能正常拉高。说明浮点乘法和浮点除法集成成功。

# 5 实验总结

在实验过程中，我主要为 VeeRwolf 的 EL2 处理器扩展了单精度浮点加、乘、除功能。通过搭建 WSL2 与相关工具环境，完成译码级信号与文件修改，将浮点运算模块集成至整数除法器，经仿真验证，浮点指令运算结果准确，原有整数除法功能正常兼容。同时编写自定义测试文件，进一步验证了新增功能的可靠性。

一开始配置环境和译码级及信号修改的步骤虽然看起来比较繁琐，但是跟着教程一步步做下来总体还是比较顺利的。但是集成的过程中难度略大，由于一开始的代码逻辑有问题，输出的波形有误，反复修改之后才实现了浮点运算的集成。总体上来说，本次实验极大地深化了我的编程和仿真调试能力，也让我体会到硬件设计的严谨性，锻炼了问题解决与实操能力。

# 6 附录代码（来自 github）

## 6.1 浮点乘法器硬件代码

```
//IEEE Floating Point Multiplier (Single Precision)
//Copyright (C) Jonathan P Dawson 2013
//2013-12-12
module multiplier(
        input_a,
        input_b,
        input_a_stb,
        input_b_stb,
        output_z_ack,
        clk,
        rst,
        output_z,
        output_z_stb,
```

```
        input_a_ack,
        input_b_ack);

  input     clk;
  input     rst;

  input     [31:0] input_a;
  input     input_a_stb;
  output    input_a_ack;

  input     [31:0] input_b;
  input     input_b_stb;
  output    input_b_ack;

  output    [31:0] output_z;
  output    output_z_stb;
  input     output_z_ack;

  reg       s_output_z_stb;
  reg       [31:0] s_output_z;
  reg       s_input_a_ack;
  reg       s_input_b_ack;

  reg       [3:0] state;
  parameter get_a         = 4'd0,
            get_b         = 4'd1,
            unpack        = 4'd2,
            special_cases = 4'd3,
            normalise_a   = 4'd4,
            normalise_b   = 4'd5,
            multiply_0    = 4'd6,
            multiply_1    = 4'd7,
            normalise_1   = 4'd8,
            normalise_2   = 4'd9,
            round         = 4'd10,
            pack          = 4'd11,
            put_z         = 4'd12;

  reg       [31:0] a, b, z;
  reg       [23:0] a_m, b_m, z_m;
  reg       [9:0] a_e, b_e, z_e;
  reg       a_s, b_s, z_s;
  reg       guard, round_bit, sticky;
  reg       [47:0] product;

  always @(posedge clk)
```

```
  begin

    case(state)

      get_a:
      begin
        s_input_a_ack <= 1;
        if (s_input_a_ack && input_a_stb) begin
          a <= input_a;
          s_input_a_ack <= 0;
          state <= get_b;
        end
      end

      get_b:
      begin
        s_input_b_ack <= 1;
        if (s_input_b_ack && input_b_stb) begin
          b <= input_b;
          s_input_b_ack <= 0;
          state <= unpack;
        end
      end

      unpack:
      begin
        a_m <= a[22 : 0];
        b_m <= b[22 : 0];
        a_e <= a[30 : 23] - 127;
        b_e <= b[30 : 23] - 127;
        a_s <= a[31];
        b_s <= b[31];
        state <= special_cases;
      end

      special_cases:
      begin
        if ((a_e == 128 && a_m != 0) || (b_e == 128 && b_m != 0)) begin
          z[31] <= 1;
          z[30:23] <= 255;
          z[22] <= 1;
          z[21:0] <= 0;
          state <= put_z;
        end else if (a_e == 128) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 255;
```

```
          z[22:0] <= 0;
          if (($signed(b_e) == -127) && (b_m == 0)) begin
            z[31] <= 1;
            z[30:23] <= 255;
            z[22] <= 1;
            z[21:0] <= 0;
          end
          state <= put_z;
        end else if (b_e == 128) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 255;
          z[22:0] <= 0;
          if (($signed(a_e) == -127) && (a_m == 0)) begin
            z[31] <= 1;
            z[30:23] <= 255;
            z[22] <= 1;
            z[21:0] <= 0;
          end
          state <= put_z;
        end else if (($signed(a_e) == -127) && (a_m == 0)) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 0;
          z[22:0] <= 0;
          state <= put_z;
        end else if (($signed(b_e) == -127) && (b_m == 0)) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 0;
          z[22:0] <= 0;
          state <= put_z;
        end else begin
          if ($signed(a_e) == -127) a_e <= -126;
          else a_m[23] <= 1;
          if ($signed(b_e) == -127) b_e <= -126;
          else b_m[23] <= 1;
          state <= normalise_a;
        end
      end

      normalise_a:
      begin
        if (a_m[23]) state <= normalise_b;
        else begin
          a_m <= a_m << 1;
          a_e <= a_e - 1;
        end
      end
```

```

        normalise_b:
        begin
          if (b_m[23]) state <= multiply_0;
          else begin
            b_m <= b_m << 1;
            b_e <= b_e - 1;
          end
        end

        multiply_0:
        begin
          z_s <= a_s ^ b_s;
          z_e <= a_e + b_e + 1;
          product <= a_m * b_m;
          state <= multiply_1;
        end

        multiply_1:
        begin
          z_m <= product[47:24];
          guard <= product[23];
          round_bit <= product[22];
          sticky <= (product[21:0] != 0);
          state <= normalise_1;
        end

        normalise_1:
        begin
          if (z_m[23] == 0) begin
            z_e <= z_e - 1;
            z_m <= z_m << 1;
            z_m[0] <= guard;
            guard <= round_bit;
            round_bit <= 0;
          end else state <= normalise_2;
        end

        normalise_2:
        begin
          if ($signed(z_e) < -126) begin
            z_e <= z_e + 1;
            z_m <= z_m >> 1;
            guard <= z_m[0];
            round_bit <= guard;
            sticky <= sticky | round_bit;
```

```
        end else state <= round;
      end

      round:
      begin
        if (guard && (round_bit | sticky | z_m[0])) begin
          z_m <= z_m + 1;
          if (z_m == 24'hffffff) z_e <=z_e + 1;
        end
        state <= pack;
      end

      pack:
      begin
        z[22 : 0] <= z_m[22:0];
        z[30 : 23] <= z_e[7:0] + 127;
        z[31] <= z_s;
        if ($signed(z_e) == -126 && z_m[23] == 0)
          z[30 : 23] <= 0;
        if ($signed(z_e) > 127) begin
          z[22 : 0] <= 0;
          z[30 : 23] <= 255;
          z[31] <= z_s;
        end
        state <= put_z;
      end

      put_z:
      begin
        s_output_z_stb <= 1;
        s_output_z <= z;
        if (s_output_z_stb && output_z_ack) begin
          s_output_z_stb <= 0;
          state <= get_a;
        end
      end

    endcase

    if (rst == 1) begin
      state <= get_a;
      s_input_a_ack <= 0;
      s_input_b_ack <= 0;
      s_output_z_stb <= 0;
    end

```

```
  end

  assign input_a_ack = s_input_a_ack;
  assign input_b_ack = s_input_b_ack;
  assign output_z_stb = s_output_z_stb;
  assign output_z = s_output_z;

endmodule
```

Listing 3: 浮点乘法器模块

## 6.2 浮点除法器硬件代码

```
//IEEE Floating Point Divider (Single Precision)
//Copyright (C) Jonathan P Dawson 2013
//2013-12-12
//
module divider(
        input_a,
        input_b,
        input_a_stb,
        input_b_stb,
        output_z_ack,
        clk,
        rst,
        output_z,
        output_z_stb,
        input_a_ack,
        input_b_ack);

  input     clk;
  input     rst;

  input     [31:0] input_a;
  input     input_a_stb;
  output    input_a_ack;

  input     [31:0] input_b;
  input     input_b_stb;
  output    input_b_ack;

  output    [31:0] output_z;
  output    output_z_stb;
  input     output_z_ack;

```

```
reg       s_output_z_stb;
reg       [31:0] s_output_z;
reg       s_input_a_ack;
reg       s_input_b_ack;

reg       [3:0] state;
parameter get_a         = 4'd0,
          get_b         = 4'd1,
          unpack        = 4'd2,
          special_cases = 4'd3,
          normalise_a   = 4'd4,
          normalise_b   = 4'd5,
          divide_0      = 4'd6,
          divide_1      = 4'd7,
          divide_2      = 4'd8,
          divide_3      = 4'd9,
          normalise_1   = 4'd10,
          normalise_2   = 4'd11,
          round         = 4'd12,
          pack          = 4'd13,
          put_z         = 4'd14;

reg       [31:0] a, b, z;
reg       [23:0] a_m, b_m, z_m;
reg       [9:0] a_e, b_e, z_e;
reg       a_s, b_s, z_s;
reg       guard, round_bit, sticky;
reg       [50:0] quotient, divisor, dividend, remainder;
reg       [5:0] count;

always @(posedge clk)
begin

  case(state)

    get_a:
    begin
      s_input_a_ack <= 1;
      if (s_input_a_ack && input_a_stb) begin
        a <= input_a;
        s_input_a_ack <= 0;
        state <= get_b;
      end
    end

    get_b:
```

```
        begin
          s_input_b_ack <= 1;
          if (s_input_b_ack && input_b_stb) begin
            b <= input_b;
            s_input_b_ack <= 0;
            state <= unpack;
          end
        end

        unpack:
        begin
          a_m <= a[22 : 0];
          b_m <= b[22 : 0];
          a_e <= a[30 : 23] - 127;
          b_e <= b[30 : 23] - 127;
          a_s <= a[31];
          b_s <= b[31];
          state <= special_cases;
        end

        special_cases:
        begin
          if ((a_e == 128 && a_m != 0) || (b_e == 128 && b_m != 0)) begin
            z[31] <= 1;
            z[30:23] <= 255;
            z[22] <= 1;
            z[21:0] <= 0;
            state <= put_z;
          end else if ((a_e == 128) && (b_e == 128)) begin
            z[31] <= 1;
            z[30:23] <= 255;
            z[22] <= 1;
            z[21:0] <= 0;
            state <= put_z;
          end else if (a_e == 128) begin
            z[31] <= a_s ^ b_s;
            z[30:23] <= 255;
            z[22:0] <= 0;
            state <= put_z;
            if ($signed(b_e == -127) && (b_m == 0)) begin
              z[31] <= 1;
              z[30:23] <= 255;
              z[22] <= 1;
              z[21:0] <= 0;
              state <= put_z;
            end
```

```
        end else if (b_e == 128) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 0;
          z[22:0] <= 0;
          state <= put_z;
        end else if (($signed(a_e) == -127) && (a_m == 0)) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 0;
          z[22:0] <= 0;
          state <= put_z;
          if (($signed(b_e) == -127) && (b_m == 0)) begin
            z[31] <= 1;
            z[30:23] <= 255;
            z[22] <= 1;
            z[21:0] <= 0;
            state <= put_z;
          end
        end else if (($signed(b_e) == -127) && (b_m == 0)) begin
          z[31] <= a_s ^ b_s;
          z[30:23] <= 255;
          z[22:0] <= 0;
          state <= put_z;
        end else begin
          if ($signed(a_e) == -127) a_e <= -126;
          else a_m[23] <= 1;
          if ($signed(b_e) == -127) b_e <= -126;
          else b_m[23] <= 1;
          state <= normalise_a;
        end
      end

      normalise_a:
      begin
        if (a_m[23]) state <= normalise_b;
        else begin
          a_m <= a_m << 1;
          a_e <= a_e - 1;
        end
      end

      normalise_b:
      begin
        if (b_m[23]) state <= divide_0;
        else begin
          b_m <= b_m << 1;
          b_e <= b_e - 1;
```

```
        end
      end

      divide_0:
      begin
        z_s <= a_s ^ b_s;
        z_e <= a_e - b_e;
        quotient <= 0;
        remainder <= 0;
        count <= 0;
        dividend <= a_m << 27;
        divisor <= b_m;
        state <= divide_1;
      end

      divide_1:
      begin
        quotient <= quotient << 1;
        remainder <= remainder << 1;
        remainder[0] <= dividend[50];
        dividend <= dividend << 1;
        state <= divide_2;
      end

      divide_2:
      begin
        if (remainder >= divisor) begin
          quotient[0] <= 1;
          remainder <= remainder - divisor;
        end
        if (count == 49) state <= divide_3;
        else begin
          count <= count + 1;
          state <= divide_1;
        end
      end

      divide_3:
      begin
        z_m <= quotient[26:3];
        guard <= quotient[2];
        round_bit <= quotient[1];
        sticky <= quotient[0] | (remainder != 0);
        state <= normalise_1;
      end

```

```
      normalise_1:
      begin
        if (z_m[23] == 0 && $signed(z_e) > -126) begin
          z_e <= z_e - 1;
          z_m <= z_m << 1;
          z_m[0] <= guard;
          guard <= round_bit;
          round_bit <= 0;
        end else state <= normalise_2;
      end

      normalise_2:
      begin
        if ($signed(z_e) < -126) begin
          z_e <= z_e + 1;
          z_m <= z_m >> 1;
          guard <= z_m[0];
          round_bit <= guard;
          sticky <= sticky | round_bit;
        end else state <= round;
      end

      round:
      begin
        if (guard && (round_bit | sticky | z_m[0])) begin
          z_m <= z_m + 1;
          if (z_m == 24'hffffff) z_e <=z_e + 1;
        end
        state <= pack;
      end

      pack:
      begin
        z[22 : 0] <= z_m[22:0];
        z[30 : 23] <= z_e[7:0] + 127;
        z[31] <= z_s;
        if ($signed(z_e) == -126 && z_m[23] == 0) z[30 : 23] <= 0;
        if ($signed(z_e) > 127) begin
          z[22 : 0] <= 0;
          z[30 : 23] <= 255;
          z[31] <= z_s;
        end
        state <= put_z;
      end

      put_z:
```

```
      begin
        s_output_z_stb <= 1;
        s_output_z <= z;
        if (s_output_z_stb && output_z_ack) begin
          s_output_z_stb <= 0;
          state <= get_a;
        end
      end

    endcase

    if (rst == 1) begin
      state <= get_a;
      s_input_a_ack <= 0;
      s_input_b_ack <= 0;
      s_output_z_stb <= 0;
    end

  end

  assign input_a_ack = s_input_a_ack;
  assign input_b_ack = s_input_b_ack;
  assign output_z_stb = s_output_z_stb;
  assign output_z = s_output_z;

endmodule
```

Listing 4: 浮点除法器模块